

Assignment: Designing a Cost-Aware Compaction Strategy for Apache Iceberg

Imagine you're working with a large-scale data platform that stores data as **partitioned collections of files**. Each partition represents a time window or category of data (e.g., logs from a given day). Over time, the platform accumulates **many small files**, which makes queries slower and costlier due to overhead per file. To solve this, the system periodically **rewrites (compacts) files** into larger, optimized ones.

But here's the challenge:

- **Rewriting is expensive** (compute + time).
- **We have a limited daily budget** (e.g., 50 compute hours).
- Not all partitions contribute equally to performance degradation.

Your task is to design an algorithm that **selects which partitions to compact**, and **how to compact them**, in order to maximize **system performance** within the budget.

Input Data (download

here):[https://datazip.atlassian.net/wiki/external/YTlxMDdkOWJkZGRhNGU1ODg5ZGRjNjY5NTFIZDA0NzI#Input-Data-\(download-here\)](https://datazip.atlassian.net/wiki/external/YTlxMDdkOWJkZGRhNGU1ODg5ZGRjNjY5NTFIZDA0NzI#Input-Data-(download-here))

 Sample JSON Dataset below

Each row in the dataset represents one partition and includes:

- `file_count`: Number of files in the partition
- `avg_file_size_mb`: Average size of files
- `delete_file_count`: Number of logical deletions that must be applied during reads
- `delete_ratio`: `delete_file_count / file_count`
- `avg_delete_file_size_mb`: Average size of delete files
- `partition_access_frequency`: How often this partition is queried

- `small_files_count`: Number of files with size less than 128mb.
-

Problem as Optimization

You must decide, for each partition:

- Whether to **compact** it or not
- Such that:
 - **Total cost $\leq$ budget**
 - **Performance gain is maximized**

Key Concepts to Use

You are free to define a **scoring function** or **statistical model** using any or all of:

- Performance cost penalty due to small files: e.g., inversely proportional to `avg_file_size` and proportional to `small_files_count`
- Performance cost penalty due to logical deletes: e.g., proportional to `delete_file_count * avg_delete_file_size_mb`
- Access-weighted penalty: partitions with high `partition_access_frequency` matter more
- Compaction cost: proportional to `file_count * avg_file_size_mb * avg_delete_file_size_mb`

Assumptions

- **Files < 128 MB** are considered small
 - **Each delete file increases query time**
 - **Compaction cost** = `file_count * avg_file_size_mb * avg_delete_file_size_mb * cost_factor`
 - Where:
 - `file_count * avg_file_size_mb` = estimated total data to read
 - `avg_delete_file_size_mb` = complexity added by applying deletes
 - `cost_factor` = normalization constant (0.001) to scale to minutes
 - **Daily compute budget** = 50 compute-hours (i.e., 3000 minutes)
-

Inputs & Hints

You can use the following as your base hyperparameters (but feel free to redefine as some might not be useful for this problem statement):

Parameter	Description
<code>target_file_size_mb</code>	Ideal file size post-compaction
<code>min_file_size_mb</code>	Minimum size to consider for compaction
<code>delete_ratio_threshold</code>	If delete files exceed this ratio, compact
<code>max_daily_cost</code>	Daily compute budget
<code>read_latency_penalty_per_file</code>	A synthetic function estimating Performance cost added by small files
<code>partition_access_frequency</code>	Query hit count per partition

Suggested observability metrics (provided as JSON below):

- Partition ID
 - File count
 - Avg file size
 - Delete file count
 - Read query frequency
 - `small_file_count`
-

Your Deliverables

1. **Optimization Model Design**

- Define a function for performance penalty
- Define a cost model per compaction
- Define selection logic or a ranking/scoring algorithm
- Explain how your algorithm decides which partitions to compact, and how

2. **(Optional Bonus) Code Simulation** (in Python, R, etc.)

- Simulate your algorithm on the sample dataset
- Output: List of partitions selected for compaction, strategy chosen, total cost used, expected performance gain

3. **Slides or 1-pager**

- Summary of your approach
 - Tradeoffs considered
 - Results of your simulation (if done)
-

Goal

Your model should recommend a set of actions (compaction decisions) for the 20 partitions in the sample data such that:

- You **stay under budget** (compute hours, or a synthetic cost function)
 - You **maximize performance gain** (reduce penalties caused by small files, delete files, and access skew)
-

Bonus Explorations

If you're feeling creative:

- Extend the model to be dynamic over time (e.g., weekly schedule)
 - Propose how to learn the best thresholds or scoring model from data
 - Suggest a real-world analogy or field (e.g., caching, load balancing) where similar logic applies
-

Evaluation Criteria

- Clarity and rigor of your proposed model
- Ability to reason about cost vs. performance trade-offs
- Creativity and feasibility of the compaction triggering logic
- Communication quality (writing, visuals)
- Bonus: Code quality, if you choose to prototype